

现代编程思想

哈希表与闭包

月兔公开课课程组

闭包 Closure

- 闭包：函数 + 它捕获的周边环境状态的引用
- 环境状态由词法作用域决定（代码结构决定，非运行时决定）

```
1. test {  
2.   let mut i = 2  
3.   fn debug_i() -> Int {  
4.     i // 捕获外层的 i  
5.   }  
6.   i = 3  
7.   inspect(debug_i(), content="3")  
8.   let i = 4 // 不同的 i  
9.   inspect(debug_i(), content="3") // 不会被新的 i 影响  
10. }
```

闭包的应用：数据封装

- 数据隐藏：外部无法直接访问内部变量
- 行为控制：可以添加校验逻辑
- 接口简洁：只暴露必要的操作

```
1. fn natural_number_get_and_set() -> (() -> Int, (Int) -> Unit) {  
2.     let mut i = 0 // 私有变量  
3.     fn get() -> Int {  
4.         i  
5.     }  
6.     fn set(new_value : Int) -> Unit {  
7.         if new_value >= 0 {  
8.             i = new_value // 只接受非负值  
9.         }  
10.    }  
11.    (get, set)  
12. }
```

数据封装示例

- 数据隐藏：外部无法直接访问内部变量
 - 行为控制：可以添加校验逻辑
 - 接口简洁：只暴露必要的操作

```
1. test {  
2.   let (get, set) = natural_number_get_and_set()  
3.   set(10)  
4.   inspect(get(), content="10")  
5.   set(-100) // 非法输入, 被忽略  
6.   inspect(get(), content="10") // 值不变  
7.   set(20)  
8.   inspect(get(), content="20")  
9. }
```

闭包与抽象：Map 接口

- 使用闭包封装不同的哈希表实现
- 对使用者而言，实现细节不重要

```
1. struct Map[K, V] {  
2.   get : (K) -> Option[V]  
3.   put : (K, V) -> Unit  
4.   remove : (K) -> Unit  
5.   size : () -> Int  
6. }
```

- 可以提供多个构造函数：
 - `Map::hash_open()` 开放寻址实现
 - `Map::hash_bucket()` 直接寻址实现

Map 实现：直接寻址

```
1. fn[K : Hash + Eq, V] Map::hash_bucket() -> Map[K, V] {
2.   let initial_length = 10
3.   let values : Array[Bucket[Entry[K, V]]] = Array::make(
4.     initial_length,
5.     { val: None },
6.   )
7.   for i = 0; i < initial_length; i = i + 1 {
8.     values[i] = { val: None }
9.   }
10.  let mut size_count = 0
11.  let length = initial_length
12.  let map = { values, length, size }
13.
14.  fn get(key : K) -> Option[V] { ... }
15.  fn put(key : K, value : V) -> Unit { ... }
16.  fn remove(key : K) -> Unit { ... }
17.  fn size() -> Int { size_count }
18.
19.  { get, put, remove, size } // 返回包含闭包的结构体
20. }
```

Map 扩展方法

```
1. fn[K, V] Map::is_empty(self : Map[K, V]) -> Bool {  
2.     (self.size)() == 0  
3. }  
4.  
5. fn[K, V] Map::contains(self : Map[K, V], key : K) -> Bool {  
6.     match (self.get)(key) {  
7.         Some(_) => true  
8.         None => false  
9.     }  
10. }
```

- 基于基本操作实现更多便利方法
- 一次定义，所有实现都可使用

Map 使用示例

```
1. test {  
2.   let map : Map[Int, Int] = Map::hash_bucket() // or Map::hash_open()  
3.  
4.   inspect(map.is_empty(), content="true")  
5.   (map.put)(1, 10)  
6.   inspect((map.get)(1), content="Some(10)")  
7.   inspect(map.is_empty(), content="false")  
8.   (map.remove)(1)  
9.   inspect(map.contains(1), content="false")  
10. }
```


回顾：内部迭代器和外部迭代器

- 内部迭代器：
 - 调用者提供回调函数，迭代过程由迭代器自身来执行
 - 无法由调用者提前中止迭代
 - `int_iter.each(fn(elem) { f(elem) })`
- 外部迭代器：
 - 调用者主动从迭代器一个个拉取元素
 - 调用者可以随时停止来取元素来中止迭代
 - `while ext_iter.next() is Some(elem) { f(elem) }`

外部迭代器

- 核心思想：调用者主动从迭代器拉取元素
 - 调用者控制何时获取下一个元素
 - 可以随时停止迭代

```
1. // 外部迭代器的使用方式
2. while iter.next() is Some(elem) {
3.     // 处理元素
4.     if <某个条件> {
5.         break // 可以随时停止!
6.     }
7. }
```

- 实现方法：使用闭包捕获迭代状态

外部迭代器：数据结构

- 本质上，外部迭代器 `Iterator[T]` 是一个函数： `() -> Option[T]`，通过 `iterator.next()` 调用
- 核心思想：
 - `next()`：每次调用返回下一个元素
 - 返回 `Some(value)` 表示还有元素
 - 返回 `None` 表示迭代结束
 - 函数内部通过闭包捕获迭代状态

外部迭代器：列表

```
1. fn[T] List::iterator(self : List[T]) -> Iterator[T] {  
2.   let mut current = self // 捕获可变状态  
3.   fn next() -> Option[T] {  
4.     match current {  
5.       Nil => None  
6.       Cons(head, tail) => {  
7.         current = tail // 更新状态  
8.         Some(head)  
9.       }  
10.    }  
11.  }  
12.  Iterator::new(next) // 生成迭代器  
13. }
```

- 外部迭代器的构造函数：

```
fn[T] Iterator::new(next : () -> Option[T]) -> Iterator[T]
```

外部迭代器：使用示例

```
1. test {  
2.   let list = Cons(1, Cons(2, Cons(3, Nil)))  
3.   let iter = list.iterator()  
4.   inspect(iter.next(), content="Some(1)")  
5.   inspect(iter.next(), content="Some(2)")  
6.   inspect(iter.next(), content="Some(3)")  
7.   inspect(iter.next(), content="None")  
8.   inspect(iter.next(), content="None")  
9.   // or  
10.  while iter.next() is Some(v) {  
11.    println(v)  
12.  }  
13. }
```

- 调用者完全控制迭代过程
- 可以在任何时候停止调用迭代器

转换迭代器

```
1. fn[T, R] Iterator::map(self : Iterator[T], f : (T) -> R) -> Iterator[R] {
2.   fn next() -> Option[R] {
3.     match self.next() {
4.       Some(value) => Some(f(value))
5.       None => None
6.     }
7.   }
8.   Iterator::new(next)
9. }
10.
11. test {
12.   let list = Cons(1, Cons(2, Cons(3, Nil)))
13.   let iter = list.iterator().map(fn(x) { x * 2 })
14.   inspect(iter, content="[2, 4, 6]")
15. }
```

转换迭代器

- 迭代器的常见用法是经过多轮转换，最终得到需要的结果
- 以下函数使得迭代器转换为新的迭代器：

```
1. fn[T, R] Iterator::map(self : Iterator[T], f : (T) -> R) -> Iterator[R]
2. fn[T] Iterator::filter(self : Iterator[T], p : (T) -> Bool) -> Iterator[T]
3. fn[T] Iterator::take(self : Iterator[T], n : Int) -> Iterator[T]
4. fn[T] Iterator::concat(self : Iterator[T], other : Iterator[T]) -> Iterator[T]
```

无穷迭代器

```
1. fn naturals() -> Iterator[Int] {  
2.     let mut n = 0  
3.     fn next() -> Option[Int] {  
4.         let current = n  
5.         n = n + 1  
6.         Some(current)  
7.     }  
8.     Iterator::new(next)  
9. }
```

- 与 take 组合使用

```
1. test {  
2.     let first_five = naturals().take(5)  
3.     inspect(first_five, content="[0, 1, 2, 3, 4]")  
4. }
```


无穷迭代器

```
1. fn naturals_external() -> Iterator[Int] {
2.   let mut n = 0
3.   fn next() -> Option[Int] {
4.     let current = n
5.     n = n + 1
6.     Some(current)
7.   }
8.   Iterator::new(next)
9. }
10.
11. let naturals_internal : Iter[Int] = Iter::new(fn(yield_) {
12.   fn go(n) {
13.     guard yield_(n) is IterContinue else { return IterEnd }
14.     go(n + 1)
15.   }
16.   go(0)
17. })
```

小结

- 哈希表
- 闭包
 - 函数 + 捕获的环境
 - 应用：数据封装、行为抽象
 - 实现外部迭代器
- 推荐阅读
 - 《算法导论》第十一章